

QualiJournal 개발 파일 사전 (Development File Dictionary)

본 문서는 QualiJournal 관리자 시스템 프로젝트의 프론트엔드, 백엔드, 백업 스크립트, 그리고 QA/테스트 자동화 스크립트들을 파일별로 정리한 개발 파일 사전입니다. 이전 구조(a.zip)와 개선된 최신 구조(b.zip)의 코드 전체를 기반으로 작성되었으며, 파일별 주요 기능과 변경사항(삭제/통합/변경), 관련된 API 호출, 그리고 의존 모듈을 간략히 설명합니다. **파일 경로는 상대 경로 기준으로 정렬되어 있으며**, 파일 이름 뒤 [old] 또는 [new] 표시는 각각 과거 구조에만 존재하거나 최신 구조에서 새로 추가된 파일임을 의미합니다.

개발 파일 목록 및 설명

- `auth_utils.py` [new] - 관리자 인증/인가 관련 헬퍼 모듈로, Admin 토큰(JWT 등) 검증과 2단계 인증(2FA) 처리, 감사 로깅 등의 보안 기능을 모아놓은 파일입니다. FastAPI 의존성으로 토큰 유효성을 확인하거나, 보안 관련 유틸리티 함수를 제공합니다. 기존 구조에서는 `common_utils.py` 등에 포함되었던 인증 로직을 분리한 것으로, 다른 모듈에서도 참조됩니다. **호출 API:** (해당 없음; 백엔드 내부 모듈) **의존성:** FastAPI, `qj_paths` (경로 유틸) 등 보안 관련 라이브러리.
- `common_utils.py` [old] - QualiJournal에서 여러 컴포넌트들이 공통으로 사용하는 유틸리티 함수들을 모아둔 모듈입니다. 예를 들어 문자열 처리, JSON 설정 로드, 공용 상수 등이 정의되어 기존 여러 스크립트에서 중복 사용을 줄였습니다. 최신 구조에서는 이 파일이 제거되었고 기능들이 `auth_utils.py` 등 개별 모듈이나 주요 코드(`server_quali.py` 등)로 흡수되었습니다. **호출 API:** (해당 없음) **의존성:** `qj_paths.py` (경로 처리), `logging_setup.py` (로깅 설정) 등.
- `config.json` - 시스템 설정값을 저장하는 JSON 구성 파일입니다. 예를 들어 **게이트 임계값**(뉴스 발행을 위해 필요한 최소 승인 기사 수)이나 특정 기능 토글(요약/번역 기능 사용 여부) 등이 키-값 형태로 정의됩니다. 서버 기동 시 이 파일을 로드하여 설정을 적용하며, 운영 중 **슬라이더 UI**를 통해 임계값을 조절하면 해당 변경사항을 이 파일에 영구 저장합니다. **호출 API:** `/api/config/gate_required` (GET으로 설정 조회, PATCH로 업데이트) **의존성:** FastAPI 백엔드(`server_quali.py`)에서 이 파일을 경로로 읽고 쓰며 활용.
- `config_schema.py` [old] - `config.json`의 구조와 기본값을 정의한 스키마 모듈입니다. Pydantic 모델 등으로 구성 파일의 형식과 필드를 검증하고, 잘못된 값에 대한 기본 동작을 지정했습니다. 최신 코드에서는 이 별도 파일이 제거되고, 설정 로직이 간소화되어 직접 JSON을 읽거나 코드 내 상수로 처리하는 방식으로 변경되었습니다. **호출 API:** (해당 없음) **의존성:** `config.json` (구성 값) 및 Pydantic (스키마 검증 용도로 사용되었을 가능).
- `db.py` [new] - 데이터베이스 연결 및 세션 관리 모듈입니다. 환경변수 `QUALI_DB_MODE`에 따라 **Cloud SQL 등 클라우드 DB 모드**와 로컬 파일(JSON 기반) 모드를 구분하여 동작합니다. 클라우드 모드일 경우 Cloud SQL 커넥터나 SQLAlchemy를 임포트하여 연결 객체를 준비하고, 로컬 모드에서는 해당 DB 라이브러리를 불러오지 않고 JSON 파일을 사용하는 등의 처리를 합니다. 이를 통해 로컬 개발 환경에서는 DB 연결 오류 없이 동작하고, 운영 환경(Cloud Run)에서는 안전하게 Cloud DB에 연결하도록 합니다. **호출 API:** (해당 없음; `server_quali.py`에서 내부 활용) **의존성:** OS 환경변수, (클라우드 모드 시) Cloud SQL Python Connector 및 SQLAlchemy.

- `engine_core.py` [old] - 뉴스 수집 및 기사 선정의 핵심 엔진 모듈입니다. 여러 뉴스 소스에서 기사를 모으고 점수를 계산하여 **상위 기사 선정, 요약본 생성, JSON 출력**까지 담당하는 핵심 로직을 포함하고 있습니다. 과거에는 이 모듈을 통해 전체 파이프라인 처리를 했으나, 최신 구조에서는 해당 기능들이 `server_quali.py` 나 `orchestrator.py` 로 통합되거나 역할이 축소되었습니다. 결과적으로 이 파일은 제거되었으며, 주요 로직은 다른 곳에서 수행됩니다. **호출 API:** (직접 호출 없음; Orchestrator 등에서 함수로 사용) **의존성:** `common_utils.py`, `qj_paths.py` 등을 통해 경로 및 설정을 불러오고, JSON 데이터 파일(`data/` 디렉터리의 기사 목록) 입출력을 처리.
- `index.html` - **관리자 대시보드 프론트엔드** 페이지로, 관리자 UI를 구성하는 단일 HTML 파일입니다. **Vanilla JavaScript**로 작성되어 여러 버튼과 입력(UI 슬라이더 등)을 통해 백엔드 Admin API를 호출하여 기능을 수행합니다 ① ②. 페이지 로드 시 **관리자 인증 토큰**(ADMIN_TOKEN)을 요구하며, 사용자가 올바른 토큰을 입력하면 해당 토큰이 브라우저 저장소에 보관되고 이후 모든 API 요청에 `Authorization: Bearer ...` 헤더로 자동 포함됩니다 ③. 주요 기능으로 “일일 보고서 생성”, “키워드 뉴스 요약/번역”, “선정된 뉴스 요약/번역”, “Markdown/CSV 내보내기” 등을 수행하는 버튼이 있고, 각 버튼 클릭 시 백엔드 API 엔드포인트를 호출합니다. (최신 버전에서는 **UI 이벤트 처리 방식이 개선되어**, 한 번 요청을 보내면 **응답이 올 때까지 버튼을 비활성화**하여 중복 클릭을 막고 로딩중 메시지와 스피너를 표시하며 ④, 오류 발생 시 **화면에 붉은 경고 메시지**를 띄워 원인을 알려줍니다 ⑤. 예를 들어 인증 실패 시 “인증 오류: 관리 토큰을 확인하세요”와 같은 안내를 UI에 표시합니다. 이러한 UX 패치는 이전 버전의 `index.html`에서는 미흡했던 부분을 보완한 것입니다.) **호출 API:** **프론트엔드에서 Ajax (fetch) 호출하는 백엔드 REST 엔드포인트들은 다음과 같습니다** - POST `/api/report` (**일일 보고서 생성**) ⑥ ⑦, POST `/api/enrich/keyword` (**키워드 전체 기사 요약 및 번역**) ⑧ ⑨, POST `/api/enrich/selection` (**선정된 기사를 요약/번역**) ⑧ ⑨, GET `/api/export/md` 및 `/api/export/csv` (**Markdown/CSV 결과를 다운로드**) ⑩, GET/PATCH `/api/config/gate_required` (**승인 필요 기사수 임계값 조회 및 변경; 슬라이더 조작 시 호출**). **의존성:** `StaticFiles*`로 서빙되며 `server_quali.py`에 내장된 토큰 인증 미들웨어에 의존 (API 호출 시 토큰 필요). 외부 JS 프레임워크 없이 동작하며, 브라우저 LocalStorage (토큰 저장), Fetch API 등을 사용.
- `logging_setup.py` [old] - 애플리케이션의 로깅 설정을 담당하는 모듈입니다. Python `logging` 라이브러리를 사용하여 로그 출력 형식, 로그 레벨, 파일 핸들러 등을 설정했습니다. 개발/운영 시 콘솔 및 파일에 일관된 형식으로 로그를 남기기 위해 사용되었으며, `engine_core.py` 등 여러 곳에서 이 모듈을 import하여 초기 설정을 적용했습니다. 최신 구조에서는 이 파일이 제거되었고, 대신 기본 로깅 설정을 간소화하거나 Python 기본 설정을 사용하도록 변경되었습니다. **호출 API:** (해당 없음) **의존성:** Python `logging` 모듈.
- `orchestrator.py` - QualiJournal **뉴스 발행 오케스트레이션**을 수행하는 독립 실행형 Python 스크립트입니다. **전체 수집-선정-발행 파이프라인**을 자동화하며, 여러 모듈과 데이터를 연계하여 **공식 뉴스 소스와 커뮤니티 소스의 기사 수집 → JSON 병합 → 점수 부여 및 정렬 → 상위 기사 자동 승인 → 요약/번역 → 최종 보고서와 데이터 출력**까지 일련의 작업을 수행합니다. 이 스크립트는 **커맨드라인에서 수동 실행**하거나, (향후) 스케줄러에 등록하여 일일 실행할 수 있도록 설계되었습니다. 독립 동작을 위해 `common_utils.py` 나 `logging_setup.py` 등에 의존하지 않도록 구현되어 있으며, 필요한 경우 자체 함수로 대체해 **외부 모듈 없이도 동작 가능한** 형태입니다. 최신 버전에서도 유지된 핵심 모듈로서, 개발 막바지에 코드 클린업을 거쳐 안정화되었으며, 이전 구조에 비해 중복 코드 의존이 줄었습니다. **호출 API:** (해당 없음; 수동 스크립트) **의존성:** 표준 라이브러리(`os`, `json` 등) 및 `tools/` 디렉터리의 여러 스크립트들을 내부에서 호출하거나 import하여 사용합니다. 출력으로 `data/` 및 `archive/` 디렉터리의 JSON/Markdown 파일들을 생성,갱신합니다.
- `quali_admin_ops.ps1` [old] - **QualiJournal 관리자 운영 스크립트**로, Windows PowerShell 환경에서 운영자가 자주 사용하는 여러 작업을 간소화한 도구입니다. Repo 경로 설정(param으로 경로 지정) 후 빌드/배포, 기타 관련 작업 또는 기타 운영 행위를 스크립트 하나로 처리하도록 작성된 것으로 추정됩니다. (예를 들어, Git 서명 커밋 설정, PR 머지 진행 등을 도울 수 있는 Runbook 형태의 스크립트로, 실제 사용 예시는 별도 인수인계 문서에 기술됨.) 최신 구조에서는 이 파일이 포함되지 않았으며, GitHub Actions 기반 CI/CD 파이프

라인으로 전환하면서 수동 운영 스크립트의 필요성이 줄어든 것으로 보입니다. **호출 API:** (해당 없음) **의존성:** Windows PowerShell 5.1, Git CLI, GitHub CLI 등 (스크립트 내용상 개발 환경 설정과 저장소 조작에 활용).

- `qj_paths.py` [old] - QualiJournal 전용 경로 유틸리티 모듈입니다. 프로젝트 디렉터리 기준으로 데이터 파일 경로를 계산하는 함수 (`rel()`) 등을 제공하여, 다른 스크립트에서 경로를 하드코딩하지 않고 `qj_paths.rel('data/selected_articles.json')` 같은 형태로 경로를 불러올 수 있게 했습니다. 이 모듈로 **데이터/아카이브 폴더** 경로를 일관되게 참조할 수 있었습니다. 최신 코드에서는 이 파일이 제거되었거나 위치가 변경되었으며, 대신 Python `pathlib.Path` 사용 또는 각 모듈 내부에 경로 상수가 정의되는 방식으로 대체되었습니다. (`auth_utils.py` 등 일부 모듈에서는 import 구문이 남아 있으나 실제 파일은 통합된 것으로 보입니다.) **호출 API:** (해당 없음) **의존성:** `os` 및 `pathlib` 등 경로 처리 표준 라이브러리.
- `run_backup_gcs.ps1` [old] - **일일 보고서 백업 스크립트**로, Cloud Run에 배포된 Admin 서비스로부터 **최신 발행본 Markdown 및 CSV를 가져와** Google Cloud Storage(GCS)에 백업하는 역할을 합니다. 실행 시 GCP 프로젝트와 Cloud Run 서비스 정보를 바탕으로 **배포 중인 Admin 서비스 URL을 구하고**, 관리자 토큰을 헤더에 실어서 Admin API의 `/api/export/md` 및 `/api/export/csv` 엔드포인트를 호출해 각각 Markdown, CSV 파일을 응답으로 받습니다. 받은 파일을 로컬 `backup/` 폴더에 저장하고, 이어서 `gsutil`이나 `gcloud storage` 명령을 통해 지정된 GCS 버킷에 업로드하여 원격 백업을 완료합니다. 최신 구조에서는 이 스크립트가 `backup_export.ps1`로 대체되어, 기능 개선과 함께 여러 개별 스크립트로 나뉘었던 백업/내보내기 작업을 하나로 통합했습니다. **호출 API:** Cloud Run Admin 서비스의 **GET** `/api/export/md`, `/api/export/csv` (각각 Markdown/CSV 추출) **의존성:** PowerShell 5.1, `gcloud` CLI (Cloud Run 서비스 조회 및 인증), `Invoke-WebRequest` / `Invoke-RestMethod` (HTTP 요청), GCS 업로드 도구.
- `run_daily_manual.ps1` [new] - **일일 발행 작업 자동 스크립트**로, 운영자가 매일 정해진 순서의 Admin API 호출을 한 번에 수행하여 뉴스 발행을 완료하도록 도와줍니다. 이 스크립트는 Cloud Run에서 실행 중인 Admin 백엔드에 순차적으로 요청을 보내어, ① **일일 보고서 생성** → ② **모든 수집기사 요약/번역** → ③ **선정된 기사들 요약/번역** → ④ **결과물 Export 및 백업**을 자동으로 수행합니다. 내부적으로 POST `/api/report`, POST `/api/enrich/keyword`, POST `/api/enrich/selection`을 차례로 호출하고, 마지막으로 `backup_export.ps1` 스크립트를 실행하여 Markdown/CSV 추출과 GCS 백업까지 마칩니다. 이를 통해 관리자 UI에 접속하지 않고도 하루 치 발행 프로세스를 일괄 수행할 수 있으며, 주기적으로 수동 실행하거나 Windows 작업 스케줄러에 등록할 수 있습니다. **호출 API:** `/api/report`, `/api/enrich/keyword`, `/api/enrich/selection` (위 순서대로 호출) - 최종적으로 `backup_export.ps1` 내에서 `/api/export/md`, `/api/export/csv` 호출 **의존성:** PowerShell 5.1, `gcloud` CLI (Cloud Run URL 조회), 사전에 발행 중인 Cloud Run 서비스 및 ADMIN_TOKEN 값 필요.
- `run_log_report_monthly.ps1` [old] - **월간 운영 리포트 자동생성 스크립트**로, 한 달간의 운영 로그나 기사 통계를 취합하여 리포트(예: CSV 또는 Markdown)를 생성하는 용도로 추정됩니다. (예: 월간 누적 기사 수, 승인율, 사용자 로그 등을 집계) Cloud Run이나 GCP Logging API를 활용해 지난 30일간의 로그/데이터를 수집한 뒤, 로컬 파일로 저장하거나 메일로 발송하는 기능이 있었을 가능성이 있습니다. 최신 구조에서는 이 스크립트가 제공되지 않으며, 해당 기능이 필요할 경우 별도 모니터링 대시보드나 로그 분석 도구로 대체된 것으로 보입니다. **호출 API:** (추정) Cloud Run Admin 서비스의 로그 엔드포인트(`/api/logs/*`) 혹은 GCP Logging 쿼리 **의존성:** PowerShell 5.1, GCP 서비스 계정 권한 (로그 조회) 등.
- `run_log_report_weekly.ps1` [old] - **주간 운영 리포트 자동생성 스크립트**로, 위 월간 리포트 스크립트의 주간 버전입니다. 지난 7일 간의 주요 지표나 로그를 정리하여 관리자용 요약을 만드는 용도로 작성되었습니다. 월간/주간 스크립트 모두 CI 파이프라인이나 스케줄러에 의해 일정 주기로 실행하려는 의도로 보이나, 최종 프로젝트에서는 이들이 적극적으로 사용되지 않아 최신 코드에선 제외되었습니다. **호출 API:** (추정) `/api/logs/*` 또는 GCP Logging **의존성:** PowerShell 5.1, GCP Logging 권한 등 (월간 리포트 스크립트와 유사).

- `run_quali_today.ps1` [old] - 일일 뉴스 발행 작업을 실행하는 구 버전의 스크립트입니다. 주로 **로컬 개발 환경**에서 `orchestrator.py` 등의 모듈을 호출해 그날의 수집/발행 작업을 진행하는 역할을 했습니다. 예를 들어 이 스크립트는 Python 인터프리터로 `run_all.py` 나 `orchestrator.py`를 실행하거나, 필요한 경우 Windows 환경에 맞게 경로 등을 설정하여 하루치 작업을 수행했을 것으로 보입니다. 최신 구조에서는 API 중심으로 재편되면서 이 스크립트는 `run_daily_manual.ps1`로 대체되었고, 수작업으로 Python 스크립트를 돌리던 방식을 Admin API 호출 방식으로 변경했습니다. **호출 API:** (구조상 Python 스크립트 직접 실행; Admin API 호출은 없음) **의존성:** PowerShell 5.1, Python 실행환경, 프로젝트 내 Python 모듈 경로 설정 등.
- `run_weekly_report.ps1` [old] - 주간 뉴스 페이지 또는 요약 리포트를 생성하는 스크립트로 추정됩니다. QualiJournal은 “하루 한 키워드” 형식이라 주간 취합본이 필수 요소는 아니지만, 운영자가 일주일간의 뉴스를 묶어볼 수 있는 리포트를 만드려는 시도로 보입니다. 이 스크립트는 아마도 주간 아카이브를 취합하여 Markdown 리포트나 HTML 페이지를 생성했을 가능성이 있습니다. 최종 구조에서는 이 기능이 사용되지 않아 스크립트가 제거되었습니다. **호출 API:** (추정) 없거나, `tools/` 스크립트(`make_daily_report.py` 등)를 주간 모드로 여러 번 실행 **의존성:** PowerShell 5.1, Python `orchestrator.py` 등 (내부적으로 활용).
- `server_quali.py` [new] - **FastAPI 기반 백엔드 메인 어플리케이션**으로, QualiJournal 관리자용 API 엔드포인트들을 모두 제공하는 핵심 서버 파일입니다. Uvicorn/Gunicorn을 통해 Cloud Run 환경에서 실행되며, 이 모듈에서 FastAPI `app` 객체를 생성합니다. 모든 Admin API가 이 파일에 정의되어 있으며, 각 엔드포인트는 ADMIN_TOKEN 인증을 필요로 하는 **Bearer 토큰 인증 미들웨어** 하에 보호됩니다. 주요 API 라우트는 다음과 같습니다 6 8 :
- `POST /api/report` - **일일 보고서 생성** 요청으로, 현재까지 승인된 기사 목록(`selected_articles.json`)을 취합해 Markdown 형식의 데일리 뉴스를 생성합니다. 실행 시 내부적으로 Markdown 파일을 작성하고, 응답으로 생성된 리포트의 경로 또는 결과를 반환합니다 (관리자 UI에서는 이 호출 후 Export 버튼으로 파일을 다운로드) 11 .
- `POST /api/enrich/keyword` - **키워드 전체 기사 요약/번역** 요청으로, 해당일 수집된 모든 기사(`selected_keyword_articles.json`)에 대해 영문 요약문을 만들고, 설정에 따라 번역 API를 통해 한글 번역을 수행합니다. 완료 시 JSON 데이터 파일을 업데이트하며, 요약/번역된 필드들을 채워넣습니다.
- `POST /api/enrich/selection` - **선정된 기사들 요약/번역** 요청으로, 승인된 기사들(`selected_articles.json`)에 대해서만 요약 및 번역을 수행합니다. 이미 keyword 단계에서 영어 요약이 있다면 이를 활용하거나, 선택된 항목에 대해 추가 번역을 진행합니다.
- `GET /api/export/md` 및 `GET /api/export/csv` - **최종 발행본 Export** 엔드포인트로, 현재 `selected_articles.json` 내용으로 생성된 Markdown 또는 CSV 결과를 클라이언트에 내려줍니다. 호출 시점에 Markdown/CSV를 실시간 생성하며 (또는 사전에 생성된 파일이 있다면 읽어서) 응답 본문으로 제공하여, 관리자 UI에서 **파일 다운로드**를 트리거합니다.
- `GET /api/config/gate_required` & `PATCH /api/config/gate_required` - **임계값 설정 조회/변경** API로, `config.json`에 저장된 승인 필요 기사 개수를 읽거나(`GET`), 변경된 값(예: 슬라이더 입력)을 받아 업데이트합니다(`PATCH`). `PATCH`는 관리자 토큰 외에 추가 권한 체크는 없지만, 입력 값에 유효성 검증을 거쳐 `config.json`을 수정 저장합니다.
- 이밖에도 `GET /api/logs/...` (관리자용 로그 조회), `POST /api/approve-ui/start` (UI에서 수동 승인 프로세스 트리거), `POST /api/tasks/flow` 및 관련 `/api/tasks/*` (비동기 작업 큐 관리 및 **Server-Sent Events** 스트림을 통한 진행 상황 전송) 등의 부가 API가 구현되어 있습니다. 예를 들어 `/api/tasks/flow`는 백엔드에서 뉴스 수집→발행까지의 **전체 플로우를 백그라운드 태스크로 실행**하며, `/api/tasks/.../stream`을 통해 진행 로그를 SSE로 실시간 전송하는 기능이 포함되어 있습니다.

이 서버 모듈은 **정적 파일 서빙**도 담당하여, 루트 경로(`/`) 또는 `/index.html`로 요청이 오면 관리자 UI 페이지를 반환합니다 2 . 또한 `/health` 등의 헬스체크 엔드포인트, Cloud Run용 메타 엔드포인트

(`/well-known/...`)도 포함되어 운영 편의성을 높였습니다. **의존성:** FastAPI 및 Pydantic (API 모델 및 스키마), Starlette (StaticFiles 서빙), Python 내장 `subprocess` / `asyncio` (필요 시 내부 스크립트 호출 및 비동기 태스크 처리), `auth_utils.py` (토큰 인증 유틸) 및 `db.py` (DB 연결 설정), `config.json` (설정 로드). 또한 `tools/` 폴더의 스크립트들이 수행하는 로직을 내부 함수로 구현하거나, JSON 파일 입출력을 통해 동일한 효과를 내도록 구성되어 있습니다.

- `smoke.ps1` [new] - **서비스 동작 점검(스모크 테스트) 스크립트**로, 배포된 QualiJournal Admin 서비스의 상태를 빠르게 확인하는 용도로 사용됩니다. 파라미터로 서비스 이름, 리전, Admin 토큰을 받아, **Cloud Run 서비스 URL을 조회**한 후 해당 URL에 간단한 HTTP 요청을 보내봅니다. 기본적으로 인증이 필요한 `/api/status` 또는 유사 엔드포인트를 호출하여 200 OK 응답을 받는지 확인하며, 성공하면 "STATUS OK" 등의 메시지를 출력합니다. 만약 토큰 문제로 응답이 실패하면, 헤더 대신 쿼리파라미터로 토큰을 보내는 등 대체 방법으로도 시도하여 (과거 버전 호환) 응답을 점검합니다. 이 스크립트를 통해 배포 후 **Admin API 생존여부와 토큰 유효성**을 빠르게 검증할 수 있습니다. **호출 API:** GET `/api/status` (또는 `/health`) - Admin Token 헤더 포함 **의존성:** PowerShell 5.1, `gcloud` CLI (URL 조회), `Invoke-WebRequest` / `Invoke-RestMethod` (HTTP 요청).
- `tests/d3_regression_check.py` [old] - **통합 테스트 스크립트** (Pytest 기반) 중 하나로, 뉴스 선별 로직에 대한 회귀 검증을 수행합니다. 예를 들어, 특정 입력(JSON 파일 세트)을 준비한 뒤 Orchestrator나 API를 통해 결과를 생성하고, 예상되는 출력(JSON 구조나 승인된 기사 수 등)이 정확히 나오는지 테스트합니다. `d3_regression_check`라는 이름으로 보아 과거 Day3 등의 시나리오나 3일치 데이터를 검사했던 것으로 추측됩니다. 이 테스트는 CI 파이프라인에서 자동 실행되어, 코드 변경으로 인해 선별 기준이나 데이터 구조에 문제가 생기지 않았는지 검증했습니다¹². 최신 구조에서는 테스트 코드가 일부 수정/교체되어 이 파일이 제거되었거나, 기능이 `tools/d3_regression_check.py`로 대체되었습니다. **호출 API:** 테스트 중에 FastAPI `TestClient`로 `/api/report` 등 주요 API를 호출하고 응답(JSON, 파일 존재 여부 등)을 검증했을 것입니다¹³¹⁴. **의존성:** Pytest, FastAPI `TestClient`, 임시 파일 시스템 (`tmp_path`) 등을 사용. ADMIN_TOKEN 필요 시 `os.environ`으로 주입하여 테스트.
- `tools/approve_top20.py` - **상위 20개 기사 일괄 승인** 스크립트입니다. `selected_keyword_articles.json` (작업본)에서 점수 등 정렬 기준으로 **상위 20개 기사**를 자동으로 `approved=true`로 표시합니다¹⁵. 편집자가 일일이 선택하지 않아도 임계 개수만큼 기사를 확보하기 위한 목적으로 사용됩니다. 이 스크립트는 수동 실행하거나 Orchestrator 흐름 중에 필요 시 호출됩니다. **호출 API:** (직접 호출 없음; Orchestrator나 운영자가 수동으로 실행) **의존성:** `data/selected_keyword_articles.json` (입력), `json` 모듈.
- `tools/augment_from_official_pool.py` - **공식 뉴스풀 보완** 스크립트입니다. 커뮤니티 소스 기사만으로 충분한 승인 기사가 확보되지 않았을 때, **공식 뉴스풀**에서 추가 기사를 선정하여 전체 기사 수를 늘려주는 역할을 합니다¹⁶. 예를 들어 승인된 기사 수가 10개뿐이라면, 이 스크립트가 공식뉴스 데이터에서 상위 5~10개 기사를 추려 `selected_keyword_articles.json`에 추가하고, 승인 기사 수도 목표치에 도달하도록 합니다. **호출 API:** 없음 (오프라인 유틸) **의존성:** 공식 뉴스 풀 데이터 (사전에 준비된 JSON 등), `selected_keyword_articles.json`.
- `tools/classify_and_rank.py` - 수집된 기사들을 분류하고 점수를 매겨 랭크를 산정하는 스크립트입니다. 기사별로 카테고리를 분류하거나 키워드 매칭을 하고, 별도의 점수 산정 알고리즘(예: 인기도, 최신성 가중치)을 적용해 정렬 우선순위를 계산합니다. Orchestrator 단계에서 초기 **기사 필터링 및 정렬**에 활용되며, 결과는 `selected_keyword_articles.json`에 `score` 필드 등으로 반영됩니다. **호출 API:** 없음 **의존성:** 분류 기준 데이터(예: 키워드 사전)와 내부 알고리즘, `json` 모듈.
- `tools/cleanup_legacy.py` - **레거시 데이터 정리** 스크립트로, JSON 구조 변경 등으로 인해 발생한 구 버전 데이터의 불일치를 해소합니다. 예를 들어 과거 버전의 `selected_articles.json`에 필드 누락이

나 형식 오류가 있으면 이를 새로운 스키마에 맞게 수정하거나, 사용되지 않는 임시 파일을 삭제하는 등의 작업을 수행합니다. 주로 업그레이드 과정에서 한번 실행해주는 용도로 작성되었습니다. **호출 API:** 없음 **의존성:** 파일 시스템 접근, JSON 모듈.

- `tools/d3_regression_check.py` - **데이터 회귀 검증** 스크립트로, 테스트용으로 작성된 별도의 실행 스크립트입니다. (Pytest를 통하지 않고) 단독 실행하여 Orchestrator 또는 엔진의 결과를 비교 검증하는데 사용됩니다. 예컨대, 며칠 간의 archive 데이터를 입력으로 주고 현재 로직으로 생성한 결과와 과거 정상 결과를 비교해 차이가 없음을 확인합니다. CI 자동화 전 개발단계에서 수동으로 결과를 살펴보는 용도로 작성되었습니다. (이름이 tests 디렉터리의 동일 이름 파일과 겹치는데, 개발 단계 구분 없이 활용된 것으로 보입니다.) **호출 API:** 없음 (내부 로직에서 Orchestrator 함수 호출) **의존성:** `archive/` 폴더 내 과거 결과물, `json` 모듈.
- `tools/enrich_cards.py` - **뉴스 카드 요약 및 번역** 스크립트입니다. 수집이 완료된 기사 목록 (`selected_keyword_articles.json`)을 입력으로 받아 각 기사에 대한 **영문 요약문**을 생성합니다¹⁷. 또한 `.env`에 설정된 번역 API 키(API_KEY)가 있으면, 생성된 영문 요약을 **한국어로 번역**하여 `summary_ko` 등의 필드로 추가 반영합니다¹⁷. 이 스크립트는 OpenAI API 등 외부 요약/번역 서비스를 호출할 수 있으며, 네트워크 요청과 비용이 발생하므로 필요시에만 수동 실행하거나, Admin UI의 “요약/번역” 버튼 동작 시 서버에서 이 로직을 수행합니다. **호출 API:** 없음 (Admin API에서 동등 기능 구현됨) **의존성:** OpenAI GPT API 또는 Papago 등 번역 API (환경설정에 따라), `selected_keyword_articles.json`.
- `tools/fix_kw_approve_min15.py` - 수집 기사 중 **최소 15개 이상이 승인되도록 강제 조정**하는 스크립트입니다. 편집자가 15개 미만으로 기사를 승인한 경우, 점수가 높은 순으로 자동 승인하여 `approved=true` 상태의 기사가 15개가 되도록 맞춥니다. (QualiJournal 발행 기준으로 **15개 이상 승인**이 권장되었기 때문에 존재하는 도구입니다.) Orchestrator에서 최종 검증 단계로 사용되어 발행본에 충분한 기사가 포함되도록 합니다. **호출 API:** 없음 **의존성:** `selected_keyword_articles.json`.
- `tools/force_approve_top40.py` - **상위 40개 기사 일괄 승인** 스크립트입니다. `approve_top20.py`와 유사하지만 더 완화된 조건으로, 기사 수를 늘리고자 할 때 사용합니다. Top 40위까지의 기사를 모두 승인 처리하여 이후 수동으로 줄이는 방식으로 활용되었을 수 있습니다. (프로젝트 초기에 실험용으로 만들어진 도구로 추정) **호출 API:** 없음 **의존성:** `selected_keyword_articles.json`.
- `tools/force_set_selected_and_approved_top20.py` - **발행본 및 승인 동시 설정** 스크립트입니다. 상위 20개 기사를 선별하여 `selected_articles.json` (발행본)에 복사하고, 이 기사들을 모두 `approved=true`로 표시하여 작업본과 발행본을 동기화합니다. 즉, “상위 20개를 승인하고 곧바로 발행 대상에 반영”하는 역할을 하며, 급하게 발행본을 구성해야 할 때 유용합니다. **호출 API:** 없음 **의존성:** `selected_keyword_articles.json` (작업본) 및 `selected_articles.json` (출력).
- `tools/make_daily_report.py` - **일일 뉴스 보고서 생성** 스크립트입니다. 최종 발행 대상 기사 목록 (`selected_articles.json`)을 불러와 당일 뉴스 요약 **Markdown 파일**을 생성합니다¹⁸. 각 기사 카드에는 제목, 출처, 요약 등이 포함되고, 미리 정의된 템플릿에 따라 본문이 작성됩니다. 생성된 Markdown은 `archive/reports/` 폴더에 해당 날짜 파일로 저장되고, 관리자 UI에서는 이 파일을 검토한 후 “Export MD/CSV” 기능으로 다운로드하거나, 이메일 발송 등에 활용합니다. **호출 API:** 없음 (Admin API `/api/report`에서 동등 기능 수행) **의존성:** `selected_articles.json`, 파일입출력, Markdown 형식 템플릿.
- `tools/merge_and_approve_top30.py` - **상위 30개 기사 병합 및 승인** 스크립트입니다. 여러 소스에서 수집된 기사 데이터를 통합하고 상위 30개를 자동 승인하여 작업본을 구성합니다. 예를 들어, 공식 뉴스와 커뮤니티 뉴스를 합쳐 총 30개를 선정하는 경우에 사용됩니다. 이 스크립트는 `rebuild_kw_from_today.py`와 `approve_top20.py`의 중간 정도 역할을 수행하며, 아마도

Top30이라는 완충적인 승인 기준을 테스트하기 위해 작성되었습니다. **호출 API:** 없음 **의존성:** 원본 데이터 JSON들 (오늘자 수집분), `selected_keyword_articles.json`.

- `tools/normalize_and_approve_top20.py` - 기사 점수 정규화 및 상위 20 승인 스크립트입니다. 수집된 기사들의 점수 편차가 클 경우 정규화를 통해 점수 분포를 조정하고, 그 후 상위 20개를 승인 처리합니다. 이를 통해 특정 소스에 점수가 치중되는 현상을 완화하고 고르게 상위권을 선정하려는 목적입니다. **호출 API:** 없음 **의존성:** `selected_keyword_articles.json`.
- `tools/promote_keyword_to_selection.py` - 특정 키워드 기사를 발행 대상으로 승격시키는 스크립트입니다. 특정 키워드를 지정하면 해당 키워드의 수집 기사들을 모두 `selected_articles.json` 발행본에 포함시키는 동작을 합니다. 일종의 수동 개입 도구로, 특정 주제가 중요해 전부 실어야 할 때 사용됩니다. (작업본 -> 발행본으로 기사들을 복사하고 `approved`도 `true`로 설정) **호출 API:** 없음 **의존성:** `selected_keyword_articles.json`, `selected_articles.json`.
- `tools/quick_publish_keyword.py` - 원터치 발행 스크립트로, 현재 키워드의 뉴스를 빠르게 발행합니다. 수집 단계부터 발행까지 한 번에 실행하며, 내부적으로 재수집 -> 요약 -> 선정 -> 보고서 생성 일련의 과정을 자동화합니다. 개발/테스트 시나 급하게 특정 키워드 뉴스를 내보낼 때 사용되며, Orchestrator를 간소한 버전으로 볼 수 있습니다. **호출 API:** 없음 **의존성:** 여러 `tools/` 모듈 (재사용) 및 `orchestrator.py` 유사 로직.
- `tools/rebuild_from_archive_force15.py` - 아카이브 복원 및 최소 15개 승인 스크립트입니다. 만약 당일 수집 데이터에 문제가 있거나 기사 수가 부족한 경우, 이전 아카이브 데이터를 참조하여 오늘자의 작업본을 구성합니다. 아카이브된 JSON들에서 최신 15개 기사를 가져와 `selected_keyword_articles.json`을 채우고 모두 승인 표시하여 발행본을 만들어냅니다. 긴급 상황에서 어제 뉴스 재사용 등의 목적으로 쓰입니다. **호출 API:** 없음 **의존성:** `archive/` 폴더 내 지난 발행 JSON들, `selected_keyword_articles.json`, `selected_articles.json`.
- `tools/rebuild_kw_from_today.py` - 당일 수집원본 리빌드 스크립트입니다. 하루 동안 여러 소스로부터 수집된 원본 JSON 파일들(예: 뉴스 API 결과 등)을 합쳐 표준 구조의 작업본(JSON)으로 생성합니다¹⁹. 각 원본 파일을 파싱하여 일관된 필드 구조로 정규화하고, `data/selected_keyword_articles.json` 파일로 합칩니다. 이는 편집자가 볼 하나의 작업본 리스트를 만드는 단계이며, Orchestrator가 가장 먼저 호출하는 스크립트 중 하나입니다. **호출 API:** 없음 **의존성:** `data/` 폴더 내 raw 수집 파일들 (파일명 규칙으로 당일자 식별), `json` 모듈.
- `tools/sync_selected_for_publish.py` - 발행본 동기화 스크립트로, 작업본 JSON에서 `approved=true`로 표시된 기사들만 추려 별도의 발행본 JSON을 갱신합니다²⁰. 구체적으로, `selected_keyword_articles.json`에서 승인된 기사들만 복사하여 `selected_articles.json`을 생성/업데이트합니다. 편집자가 관리 화면을 통해 승인을 마쳤을 때 이 스크립트를 실행하면 최종 발행 대상 파일이 준비되며, 이후 보고서 생성이나 Export 기능이 이 발행본을 기준으로 동작합니다. **호출 API:** 없음 (Admin API `/api/report` 호출 시 내부적으로 이 로직을 수행하거나 사전에 실행) **의존성:** `selected_keyword_articles.json`, `selected_articles.json`.

위에 정리된 Markdown 문서는 QualiJournal 프로젝트의 개발 파일 구조와 각 파일의 역할, 그리고 이전 대비 변경점을 망라하여 설명합니다. 아래에는 동일한 내용을 JSON 구조로 정리한 데이터가 제공됩니다 (파일 경로를 키로 하고, 각 파일의 기능/호출 API/의존성을 값 객체로 포함). JSON 객체의 키는 상대 경로 기준으로 정렬되어 있습니다.

```

{
  "auth_utils.py": {
    "function":
      "Authentication and authorization helpers (e.g. verifying ADMIN_TOKEN, potential JWT and 2FA support, and security audit logging). Centralizes security logic for the admin API, introduced in the new version to replace common_utils for auth-related functions.",
    "calls_api": null,
    "dependencies": ["FastAPI auth dependencies", "qj_paths (for path resolution)",
      "security libraries (JWT, etc.)"]
  },
  "common_utils.py": {
    "function": "Shared utility functions and config handling used across the project (string processing, JSON config loading, common constants). Aimed to avoid code duplication in older version; removed in new version with its functions integrated into other modules (e.g. auth_utils, server code).",
    "calls_api": null,
    "dependencies": ["qj_paths.py (path helper)", "logging_setup.py (logging config)", "Python json/os"]
  },
  "config.json": {
    "function": "JSON configuration file storing system settings like gate threshold (min required approved articles) and feature toggles. Loaded at server startup and updated to persist runtime changes (e.g. slider-adjusted thresholds).",
    "calls_api": "/api/config/gate_required (GET for reading settings, PATCH for updates)",
    "dependencies": ["Loaded by server_quali.py on startup", "written by server on threshold changes"]
  },
  "config_schema.py": {
    "function": "Defines schema/structure for config.json (likely via Pydantic models) to validate configuration fields and provide default values. Used in older version for config integrity.",
    "calls_api": null,
    "dependencies": ["config.json structure", "Pydantic for schema validation"]
  },
  "db.py": {
    "function": "Database connection module that conditionally sets up a Cloud SQL (or other DB) connection if QUALI_DB_MODE is \"cloud\". Ensures that in local/test mode, DB libraries are not imported (preventing errors) and uses local JSON files instead. Prepares DB session or client for use by the API if needed.",
    "calls_api": null,
    "dependencies": ["os env vars (QUALI_DB_MODE, DB credentials)", "Cloud SQL Connector & SQLAlchemy (in cloud mode)"]
  },
  "engine_core.py": {
    "function": "Core engine for news processing: collects and aggregates articles, computes scores/ranks, selects top articles, and prepares outputs (JSON summaries). In older version, orchestrated main logic for curation pipeline; removed in new version as its functionality was merged into orchestrator or server logic.",
    "calls_api": null,
    "dependencies": ["common_utils (for config/util)", "qj_paths (for file paths)", "data JSON files for input/output"]
  }
}

```

```

},
"index.html": {
  "function": "Single-page Admin dashboard UI. Provides buttons and controls to trigger backend
admin API functions (daily report generation, keyword/selection summarization, exporting
Markdown/CSV). Demands admin token on load and includes it in all AJAX calls. The new version
improved event handling (disabling buttons during processing, showing loading indicators and error
alerts to the user for better UX).",
  "calls_api": [
    "POST /api/report",
    "POST /api/enrich/keyword",
    "POST /api/enrich/selection",
    "GET /api/export/md",
    "GET /api/export/csv",
    "GET/PATCH /api/config/gate_required"
  ],
  "dependencies": ["Served by FastAPI StaticFiles in server_quali.py", "Vanilla JS (Fetch API,
LocalStorage)"]
},
"logging_setup.py": {
  "function": "Configures Python logging (format, level, handlers) for the application. Imported by
various scripts in older version to ensure consistent logging behavior. In the new version, this
standalone setup was removed and logging is handled more simply or by defaults.",
  "calls_api": null,
  "dependencies": ["Python logging module"]
},
"orchestrator.py": {
  "function": "Standalone orchestrator script coordinating the end-to-end daily news pipeline.
Executes all steps from data collection and merging, through ranking and auto-approvals, to final
Markdown report generation. Can be run manually or via scheduler as a batch job. Updated version
is self-contained (does not require common_utils/logging_setup) and handles JSON inputs/outputs
in data/archive directories.",
  "calls_api": null,
  "dependencies": ["Standard Python libs (os, json, etc.)", "tools scripts (imported or invoked for
each pipeline step)", "data/ JSON files for input/output"]
},
"quali_admin_ops.ps1": {
  "function": "Legacy PowerShell runbook for admin operations and repository management (likely
included steps for commit signing, CI checks, merging PRs, etc.). Provided a guided sequence of
commands for the operator in older version. Not present in the new version, as CI/CD automation
replaced many manual ops tasks.",
  "calls_api": null,
  "dependencies": ["PowerShell 5.1 environment", "Git & GitHub CLI (for repo operations)",
"Windows OS"]
},
"qj_paths.py": {
  "function": "Path helper module defining a function (e.g. rel()) to resolve file system paths relative
to project root (used to locate data files, archive directories consistently). Used in older code by
scripts to open JSON files without hardcoding paths. Removed in new version (path handling now
done inline or via Python pathlib).",
  "calls_api": null,
  "dependencies": ["os/pathlib for path resolution", "used by common_utils and others in old

```

```
code"]
},
"run_backup_gcs.ps1": {
  "function": "PowerShell script to backup the daily report to cloud storage (GCS). It retrieves the
current Markdown and CSV report from the running admin service (calls /api/export/md and /api/
export/csv endpoints with proper token) and saves them locally, then uploads these files to a Google
Cloud Storage bucket. In the new version, this functionality is consolidated into
backup_export.ps1.",
  "calls_api": [
    "GET /api/export/md",
    "GET /api/export/csv"
  ],
  "dependencies": ["gcloud CLI (for Cloud Run service URL)", "Invoke-WebRequest (to call Admin
API)", "GCS CLI or gsutil (for upload)"]
},
"run_daily_manual.ps1": {
  "function": "PowerShell script to manually trigger the entire daily news publishing workflow via
API calls. It sequentially calls the admin API endpoints to generate the daily report, perform keyword
and selection summaries, then invokes backup_export.ps1 to export and back up the results.
Automates what an operator would do step-by-step in the UI for daily publication.",
  "calls_api": [
    "POST /api/report",
    "POST /api/enrich/keyword",
    "POST /api/enrich/selection"
  ],
  "dependencies": ["gcloud CLI (to get service URL)", "Admin API (requires valid ADMIN_TOKEN)",
"Backup_export.ps1 (for final export step)"]
},
"run_log_report_monthly.ps1": {
  "function": "Automation script (PowerShell) intended to compile a monthly report of admin
activity or system logs. Likely queries logs or statistics for the past month and produces a summary
(possibly a CSV or markdown). It was part of older maintenance scripts and is not included in the
updated system (monitoring handled elsewhere or deprecated).",
  "calls_api": "/api/logs/* (or uses GCP logging API) [assumed]",
  "dependencies": ["PowerShell 5.1", "GCP Logging (for log data)", "Admin API or GCP APIs"]
},
"run_log_report_weekly.ps1": {
  "function": "Similar to the monthly log report script, but for weekly intervals. It would gather the
past 7 days of logs or stats to produce a weekly summary for admin review. Present in older version
for scheduled operations, but removed in the new version as its functionality was likely not required
or was superseded by other tools.",
  "calls_api": "/api/logs/* [assumed]",
  "dependencies": ["PowerShell 5.1", "Logging/Monitoring APIs"]
},
"run_quali_today.ps1": {
  "function": "Older daily run script executed on Windows to perform the news pipeline via direct
script calls (without using API). It probably ran orchestrator.py or related Python scripts to collect
and publish the day's news. Replaced in the new system by run_daily_manual.ps1 which uses the
deployed API instead of local Python execution.",
  "calls_api": null,
  "dependencies": ["PowerShell 5.1", "Python environment (to run orchestrator or run_all.py)",
```

```

"Project files access"]
},
"run_weekly_report.ps1": {
  "function":
    "Legacy script to generate a weekly news report or newsletter combining daily outputs. Possibly
    aggregated multiple days of news into one report. This was part of the older toolkit but was not
    carried into the new version (weekly compilation deemed unnecessary for final product or handled
    via external means).",
  "calls_api": null,
  "dependencies": ["PowerShell 5.1", "Project archive data (for multiple days)", "Markdown
    generation scripts"]
},
"server_quali.py": {
  "function": "Main FastAPI server application providing the Admin REST API. Defines all admin
    endpoints with token authentication middleware and runs the Uvicorn app. Key endpoints include:
    POST /api/report (generate daily Markdown report from approved articles), POST /api/enrich/
    keyword and /api/enrich/selection (generate summaries and translations for all collected or just
    approved articles), GET /api/export/md and /api/export/csv (download final report as Markdown or
    CSV), GET/PATCH /api/config/gate_required (read or update approval threshold). It also includes
    routes for log viewing, initiating background flows (tasks), health checks, and serves the frontend
    index.html. This file replaces the need for separate scripts by integrating their logic or invoking them
    internally, ensuring all admin functions are accessible via HTTP API.",
  "calls_api": [
    "/api/report",
    "/api/enrich/keyword",
    "/api/enrich/selection",
    "/api/export/md",
    "/api/export/csv",
    "/api/config/gate_required",
    "(and others like /api/logs/*, /api/tasks/* within the file itself)"
  ],
  "dependencies": ["FastAPI & Pydantic", "auth_utils (for token verification)", "subprocess/asyncio
    (to run tasks or scripts)", "config.json (for settings)", "data JSON files (as a lightweight DB)"]
},
"smoke.ps1": {
  "function": "Smoke test script to verify that the Admin service is up and the token works. It fetches
    the Cloud Run service URL and makes a simple authenticated request (e.g. to /api/status or /health).
    If the response is OK, it confirms the service is running and accessible. Useful after deployment to
    quickly test connectivity and auth without opening the UI.",
  "calls_api": "GET /api/status (or /health)",
  "dependencies": ["gcloud CLI (to get service URL)", "Invoke-WebRequest (HTTP GET)", "Valid
    ADMIN_TOKEN"]
},
"tests/d3_regression_check.py": {
  "function": "Pytest integration test to ensure no regressions in the news selection and report
    generation logic. It likely sets up sample data and uses FastAPI TestClient to call endpoints (like /api/
    report) then compares outputs (JSON or files) with expected results. Helps catch bugs such as
    incorrect variable references or off-by-one errors in approval logic. Removed or replaced in the new
    codebase as tests were restructured.",
  "calls_api": "Multiple admin API endpoints under test (e.g. POST /api/report, /api/enrich/*) via
    TestClient",

```

```

    "dependencies": ["Pytest framework", "FastAPI TestClient", "Temporary JSON files (for
simulation)", "ADMIN_TOKEN injection for tests"]
  },
  "tools/approve_top20.py": {
    "function": "Marks the top 20 highest-ranked articles as approved in the working set JSON.
Ensures at least 20 articles are marked approved (if available) by setting `approved=true` on the top
entries according to score or rank. Used to expedite the approval process in automation or to meet
minimum count requirements.",
    "calls_api": null,
    "dependencies": ["data/selected_keyword_articles.json (input)", "Python json handling"]
  },
  "tools/augment_from_official_pool.py": {
    "function": "If the number of approved community-sourced articles is insufficient, this script pulls
additional articles from an official news pool to boost the total. It selects some top articles from
official sources to ensure around 20 total articles with at least 15 approved ones 21. This fills gaps
when community content is lacking.",
    "calls_api": null,
    "dependencies": ["Official news pool data source (JSON)", "selected_keyword_articles.json
(merged into)"]
  },
  "tools/classify_and_rank.py": {
    "function": "Classifies and scores collected articles. Possibly groups articles by category or
keyword and computes a score for each (based on relevance, popularity, etc.), then ranks them.
Helps prepare the sorted list of articles in the daily working set for further processing (approval or
summary).",
    "calls_api": null,
    "dependencies": ["Keyword classification rules or model", "selected_keyword_articles.json",
"scoring algorithm (implemented within)"]
  },
  "tools/cleanup_legacy.py": {
    "function":
"Cleans up or migrates legacy data formats. For example, fixes old JSON files that don't match the
current schema or removes deprecated fields/files. Used during upgrades or maintenance to ensure
the data files (selected_ * JSONs) conform to expected structure in the current version.",
    "calls_api": null,
    "dependencies": ["File system access (data/ and archive/ directories)", "json module"]
  },
  "tools/d3_regression_check.py": {
    "function": "Data regression check script (similar to the test but executable as a tool). Runs the
pipeline or parts of it on historical data (possibly 3-day span or a specific scenario) and compares the
outcome to an expected result to ensure consistency. It was likely used during development to
manually verify algorithm changes.",
    "calls_api": null,
    "dependencies": ["archive historical JSON files (for comparison)", "orchestrator/engine
functions", "json and difflib for comparison"]
  },
  "tools/enrich_cards.py": {
    "function": "Generates English summaries for all articles in the working set and, if an API key is
provided, translates those summaries into Korean. This populates each article entry with summary
fields (`summary_en`, and `summary_ko` if applicable) 17. Typically run after article collection
and ranking, before final selections. It relies on external NLP/translation services.",

```

```

"calls_api": null,
"dependencies": ["OpenAI or translation API (via API_KEY)", "selected_keyword_articles.json as
input", "requests/urllib for API calls"]
},
"tools/fix_kw_approve_min15.py": {
"function": "Ensures at least 15 articles are approved in the working set. If fewer than 15 are
marked approved by the editor, this script will automatically set additional top-ranked articles to
approved until the count reaches 15. This safeguards the minimum content requirement for
publication.",
"calls_api": null,
"dependencies": ["selected_keyword_articles.json", "json module"]
},
"tools/force_approve_top40.py": {
"function": "Forcibly marks the top 40 articles as approved. Similar to approve_top20.py but with
a higher threshold, possibly used in early testing or in scenarios requiring a larger pool of approved
articles (e.g., expanding content or stress-testing).",
"calls_api": null,
"dependencies": ["selected_keyword_articles.json"]
},
"tools/force_set_selected_and_approved_top20.py": {
"function": "Takes the top 20 articles and directly sets them as the final selection, marking them
approved. In one step, it creates/overwrites selected_articles.json with these top 20 and flags them
as approved in both working and final lists. Used to quickly define the publish list without manual
approval steps.",
"calls_api": null,
"dependencies": ["selected_keyword_articles.json (source)", "selected_articles.json (output)"]
},
"tools/make_daily_report.py": {
"function": "Compiles the daily news report in Markdown using the approved articles from
selected_articles.json 18. It formats each news card with title, source, summary, and any editor
notes, then writes out a Markdown file for the day's news. The output is saved in archive/reports/
(and can be converted to HTML or used for distribution).",
"calls_api": null,
"dependencies": ["selected_articles.json (input for content)", "file system (archive/reports for
output)"]
},
"tools/merge_and_approve_top30.py": {
"function": "Merges articles from various sources and auto-approves the top 30. Likely used when
combining official and community articles: it merges datasets and ensures a broader set of 30
articles with all marked approved (or at least prepares 30 for further filtering). It provides a middle-
ground between top20 and top40 scripts, perhaps for testing different cutoff points.",
"calls_api": null,
"dependencies": ["Raw source JSON files (multiple)", "selected_keyword_articles.json (output)"]
},
"tools/normalize_and_approve_top20.py": {
"function": "Normalizes article scores then approves the top 20. If some sources' scores are
skewed, this script adjusts the scoring distribution (normalization) so that the selection isn't biased,
and then selects 20 to approve. It helps maintain fairness or consistency in what gets approved.",
"calls_api": null,
"dependencies": ["selected_keyword_articles.json", "math for normalization calculations"]
},

```

```

"tools/promote_keyword_to_selection.py": {
  "function": "Promotes all articles of a given keyword to the final selection. Essentially, for a
specified keyword (or current keyword context), it moves all entries from the working set to the
publish set and marks them approved. This ensures that particular keyword's news will all be
published, used when an editor decides every item under that topic should go out.",
  "calls_api": null,
  "dependencies": ["selected_keyword_articles.json", "selected_articles.json"]
},
"tools/quick_publish_keyword.py": {
  "function": "Runs a streamlined pipeline to publish news for the current keyword quickly. It may
combine multiple steps: fetch latest data, auto-select top articles, generate summaries, and produce
the report in one go. Useful for testing or urgent publishing of a single keyword edition without the
full review process.",
  "calls_api": null,
  "dependencies": ["orchestrator functions (internally)", "tools scripts (reused internally)", "data
JSON inputs"]
},
"tools/rebuild_from_archive_force15.py": {
  "function": "Reconstructs today's working set from archive data, ensuring 15 approved articles. In
cases of data loss or insufficient new articles, this script pulls articles from recent archives and
populates the current selected_keyword_articles.json with them (marking at least 15 as approved).
It provides a fallback to use previous content to meet publication requirements.",
  "calls_api": null,
  "dependencies": ["archive/ JSON files (recent days)", "selected_keyword_articles.json
(rewritten)", "selected_articles.json"]
},
"tools/rebuild_kw_from_today.py": {
  "function": "Merges raw collected JSON files of the day into the standard working JSON file 19 . It
reads multiple source files (from news APIs, web scrapers, etc.) from the current date and combines
them into one consolidated file: data/selected_keyword_articles.json. This forms the basis for all
further editing and processing for the day's news.",
  "calls_api": null,
  "dependencies": ["data/*(today)*.json (raw input files)", "selected_keyword_articles.json
(output)"]
},
"tools/sync_selected_for_publish.py": {
  "function": "Synchronizes the final publish JSON with the current approvals in the working JSON
20 . It filters the working list for approved=true entries and writes those to selected_articles.json
(overwriting it). This ensures that the publish set exactly matches what has been approved. Typically
run right before report generation or as part of the report API logic.",
  "calls_api": null,
  "dependencies": ["selected_keyword_articles.json (source)", "selected_articles.json
(destination)"]
}
}

```

6 8 10 13 14 1011_3QualiJournal 관리자 시스템 최종 가이드북.pdf

file:///file-6Hu18vVrjQweSb5SZz6a6M

7 9 1013_1QualiJournal 관리자 시스템 최종 통합 작업 가이드북.pdf

file:///file-6DfuAdPY3q666RS3cV5ydM